Database Management Systems

Complete University & Professional Core Study Guide

Comprehensive Coverage: Architecture, Entity-Relationship Modeling, Relational Algebra, Advanced SQL, Normalization Theory (1NF–5NF), File Organization & Indexing (B/B+ Trees), Transaction Processing, Concurrency Control Protocols, Database Recovery, and Distributed Systems / NoSQL.
Designed for: Computer Science Undergraduates, GATE Aspirants, Database Administrators, and Software Engineering Interviewees.

Document Syllabus Overview

1. **Module 1: Introduction to DBMS & Architecture** — File Systems vs. DBMS, 3-Schema Architecture, Data Independence, DBMS Roles.
2. **Module 2: Entity-Relationship (ER) Modeling** — Entities, Attributes, Relationships, Mapping Cardinalities, Weak Entities, ER-to-Relational Mapping.
3. **Module 3: Relational Data Model & Relational Algebra** — Relational Concepts, Integrity Constraints, Fundamental & Extended Operators, Tuple Relational Calculus (TRC).
4. **Module 4: Structured Query Language (SQL) Deep Dive** — DDL, DML, DQL, DCL, TCL, Aggregation, Subqueries, Joins, Window Functions, Views & Triggers.
5. **Module 5: Relational Database Design & Normalization** — Anomalies, Functional Dependencies, Closure & Keys, 1NF, 2NF, 3NF, BCNF, 4NF, 5NF, Lossless & Dependency Preservation.
6. **Module 6: File Organization, Indexing & B/B+ Trees** — Physical Storage, Heap/Sequential Files, Hashing, Index Types, B-Trees & B+ Tree Operations.
7. **Module 7: Transaction Processing & ACID Properties** — Transaction States, ACID Guarantees, Serializability, Schedule Analysis, Precedence Graphs.
8. **Module 8: Concurrency Control Mechanisms** — Lock-Based Protocols (2PL, Strict 2PL, Rigorous 2PL), Timestamp Ordering, Multiversion Concurrency Control (MVCC), Deadlock Handling.
9. **Module 9: Database Recovery Systems** — System Failures, Log-Based Recovery (WAL Protocol), Deferred vs. Immediate Update, ARIES Recovery Algorithm, Checkpointing.
10. **Module 10: Distributed Databases & NoSQL Architecture** — 2-Phase Commit (2PC), CAP Theorem, BASE Consistency, NoSQL Classifications (Document, Key-Value, Columnar, Graph).
11. **Appendix A: Comprehensive DBMS Interview & Review Questions** — Technical Q&A spanning all modules.

Module 1: Introduction to DBMS & Architecture

1.1 Fundamental Concepts

A **Database** is an organized collection of logically related data stored systematically on electronic media. A **Database Management System (DBMS)** is a comprehensive software package that acts as an interface between the database

and end-users or application programs. It provides mechanisms for defining, creating, querying, updating, and administering database structures.

1.2 Database Systems vs. File-Processing Systems

Before the advent of DBMS, organizations relied on traditional operating system file structures to manage records. File-based management had critical flaws:

- **Data Redundancy and Inconsistency:** Since independent departments maintained their own files, duplicate information (e.g., student address) was stored in multiple locations. Discrepancies naturally arose when one file was updated without propagating the changes to others.
- **Difficulty in Accessing Data:** Fetching ad-hoc reports required writing customized C or COBOL programs, making quick decision-making nearly impossible.
- **Data Isolation:** Data was scattered across multiple files with inconsistent storage formats, hindering unified querying.
- **Integrity Problems:** Data constraints (e.g., `account_balance >= 0`) were embedded in application code. Adding new constraints required modifying all existing application programs.
- **Atomicity Problems:** A system crash during a fund transfer between bank accounts could result in money being deducted from one account without being credited to the target account.
- **Concurrent Access Anomalies:** Simultaneous updates by multiple users led to lost updates or corrupted data states.
- **Security Constraints:** Restricting access to granular records or individual attributes (e.g., hiding salary details from general staff) was extremely difficult to enforce at the OS file level.

1.3 The ANSI-SPARC Three-Schema Architecture

To isolate user applications from physical storage details and provide high levels of abstraction, the ANSI-SPARC framework defines a three-tiered architecture:

Schema Level	Description	Target User / Focus
External Level (View Schema)	Describes how individual user groups perceive the data. Provides customized, tailored views of the database while hiding unneeded tables or fields.	End Users & Application Developers
Conceptual Level (Logical Schema)	Describes *what* data is stored in the database and the relationships among data entities. Defines tables, fields, constraints, security rules, and business logic.	Database Administrators (DBAs) & System Analysts
Internal Level (Physical Schema)	Describes *how* the data is physically stored on disk blocks. Specifies index types, file organizations (Heap, Sequential, Hash), page sizes, and compression algorithms.	System Designers & DB Engines

1.4 Data Independence

Data independence refers to the ability to modify the schema at one level of a database system without altering the schema at the next higher level.

- **Logical Data Independence:** The capacity to modify the conceptual schema (e.g., adding a new entity, altering column definitions) without forcing changes to existing external views or user application code.

- **Physical Data Independence:** The capacity to modify the internal/physical schema (e.g., changing file organizations, building B+ tree indexes, repartitioning disks) without requiring updates to the conceptual or external schemas.

1.5 Database Users & Roles

- **Database Administrator (DBA):** Responsible for defining schemas, managing security access, monitoring performance, performing backups/disaster recovery, and tuning indexes.
- **Database Designers:** Responsible for identifying data entities, attributes, relationships, and enforcing constraints during conceptual and logical database design phases.
- **Application Programmers:** Software developers who write code in languages like Java, Python, or C# using SQL embedded via JDBC/ODBC to interact with the database.
- **Sophisticated End Users:** Analysts who interact with the database using ad-hoc SQL queries or BI tools to perform complex data analysis.
- **Naive / Parametric Users:** End users who interact with the database through pre-built graphical user interfaces (e.g., bank tellers, e-commerce shoppers).

Module 2: Data Modeling Using Entity-Relationship (ER) Diagram

2.1 Core ER Concepts

The Entity-Relationship (ER) model is a high-level conceptual model used to represent the logical structure of a database visually.

- **Entity:** A real-world object or concept that is distinguishable from other objects (e.g., Student John, Course CS101).
- **Entity Set:** A collection of entities of the same type sharing identical attributes (e.g., all Students in a university).
- **Relationship:** An association among two or more entities (e.g., Student John *Enrolls In* Course CS101).
- **Relationship Set:** A set of relationships of the same type.

2.2 Classification of Attributes

Attribute Category	Definition	Example
Simple (Atomic)	Attribute that cannot be divided into smaller subcomponents.	`Age`, `SSN`
Composite	Attribute that can be broken down into meaningful subparts.	`Name` (First_Name, Last_Name), `Address`
Single-Valued	Holds only a single value for a given entity instance.	`Date_of_Birth`
Multi-Valued	Can hold a set of multiple values for a single entity instance.	`Phone_Numbers`, `Email_Addresses`
Derived	Value calculated dynamically from another attribute or current date.	`Age` (derived from `Date_of_Birth`)

2.3 Mapping Constraints & Cardinality Ratios

Mapping cardinality specifies the maximum number of entity instances in one entity set that can be related to entity instances in another entity set via a relationship set.

- **One-to-One (1:1):** An entity in A is associated with at most one entity in B, and vice versa (e.g., Department has one Manager).
- **One-to-Many (1:N):** An entity in A can be associated with multiple entities in B, but an entity in B is associated with at most one entity in A (e.g., Department employs multiple Employees).
- **Many-to-One (N:1):** Multiple entities in A can be associated with one entity in B (e.g., Employees work in one Department).
- **Many-to-Many (M:N):** An entity in A can be associated with any number of entities in B, and an entity in B can be associated with any number in A (e.g., Students enroll in multiple Courses).

Participation Constraints:

- **Total Participation (Double Line):** Every entity in the entity set MUST participate in at least one relationship instance (e.g., every Employee must belong to a Department).
- **Partial Participation (Single Line):** Some entities in the set MAY NOT participate in any relationship instance (e.g., not every Employee manages a Department).

2.4 Weak Entity Sets

A **Weak Entity Set** is an entity set that does not possess a primary key of its own. It depends on an **Owner (Identifying) Entity Set** for its existence.

- **Partial Key (Discriminator):** A set of attributes that uniquely distinguishes weak entities belonging to the same owner entity (e.g., `Dependent_Name` for a given Employee).
- **Identifying Relationship:** The relationship connecting a weak entity set to its owner entity set (represented by a double diamond).
- **Primary Key of Weak Entity:** Formed by combining the Primary Key of the Owner Entity Set + Partial Key of the Weak Entity Set.

2.5 ER-to-Relational Mapping Rules

Algorithm for Converting ER Diagrams into Relational Schemas

1. **Strong Entity Sets:** Create a table for each strong entity set. Include all simple attributes. For composite attributes, include only subparts.
2. **Weak Entity Sets:** Create a table for the weak entity set. Include its simple attributes + the primary key of the owner entity as a foreign key. The combined key forms the primary key.
3. **1:1 Relationship:** Include the primary key of one table as a foreign key in the other table (preferably on the total participation side).
4. **1:N Relationship:** Take the primary key from the "1" side and place it as a foreign key in the table on the "N" side.
5. **M:N Relationship:** Create a separate junction table containing the primary keys of both participating entity sets as foreign keys. Their combination forms the composite primary key.
6. **Multi-Valued Attributes:** Create a new table containing the attribute itself + the primary key of the parent entity set as a foreign key.

Module 3: Relational Data Model & Relational Algebra

3.1 Formal Relational Structure

In the relational model, data is organized into tables called **Relations**. Mathematically, a relation R over domains $D_1, D_2, ..., D_n$ is a subset of the Cartesian product $D_1 \times D_2 \times ... \times D_n$.

- **Tuple:** A row in a relation representing a record.
- **Attribute:** A column in a relation representing a property.
- **Degree (Arity):** Number of attributes in a relation.
- **Cardinality:** Number of tuples (rows) in a relation.
- **Domain:** The set of allowable atomic values for an attribute.

3.2 Relational Integrity Constraints

- **Domain Constraint:** Specifies that every value in a column must be an atomic element belonging to the declared domain.
- **Key Constraint:** No two distinct tuples in a relation can have identical values for candidate key attributes.
- **Entity Integrity Constraint:** No primary key attribute value can be NULL.
- **Referential Integrity Constraint:** A foreign key value in a referencing relation must either match a valid primary key value in the referenced relation or be NULL.

3.3 Relational Algebra Operators

Relational Algebra is a procedural query language consisting of operations that take one or two relations as input and produce a new relation as output.

1. Fundamental Operators:

Operator	Symbol	Description & Formal Definition
Selection	$\sigma_p(R)$	Filters rows in relation R that satisfy condition p. $\sigma_p(R) = \{ t \mid t \in R \wedge p(t) \}$
Projection	$\pi_{A1, ..., An}(R)$	Selects specified columns $A1, ..., An$ and automatically eliminates duplicate rows.
Union	$R \cup S$	Combines tuples from two union-compatible relations R and S. $R \cup S = \{ t \mid t \in R \vee t \in S \}$
Set Difference	$R - S$	Returns tuples in R that do not appear in S. $R - S = \{ t \mid t \in R \wedge t \notin S \}$
Cartesian Product	$R \times S$	Pairs every tuple of R with every tuple of S. Output degree = $deg(R) + deg(S)$; Cardinality = $card(R) \times card(S)$.
Rename	$\rho_{X(A1, ..., An)}(R)$	Renames the relation R to X and optionally renames attributes.

2. Derived & Join Operators:

- **Set Intersection ($R \cap S$):** Returns tuples present in both R and S. Expressed as $R \cap S = R - (R - S)$.

- **Theta Join ($R \bowtie_\theta S$):** Performs Cartesian product followed by selection based on predicate θ:

$$R \bowtie_\theta S = \sigma_\theta(R \times S)$$

- **Natural Join ($R \bowtie S$):** Equi-join on all attributes having common names in both relations, projecting out duplicate join attributes.
- **Division Operator ($R \div S$):** Used for queries containing universal quantification ("for all", "every"). Let R have schema (A, B) and S have schema (B). $R \div S$ returns tuples a such that for all tuples $b \in S$, $(a, b) \in R$.

$$R \div S = \pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$$

Relational Algebra Worked Example

Given relations `Student(sid, sname, rating, age)` and `Reserves(sid, bid, day)`:

Query: Find names of students who have reserved Boat 103.

$$\pi_{sname}(\sigma_{bid = 103}(Reserves) \bowtie Student)$$

Module 4: Structured Query Language (SQL) Deep Dive

4.1 SQL Schema Definition (DDL)

Data Definition Language (DDL) manages schema structures. Key statements include `CREATE`, `ALTER`, `DROP`, and `TRUNCATE`.

```
-- Complete DDL Setup with Tables and Constraints
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE,
    location VARCHAR(100)
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    salary DECIMAL(10, 2) CHECK (salary > 0),
    dept_id INT,
    manager_id INT,
    hire_date DATE DEFAULT CURRENT_DATE,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id) ON DELETE SET NULL,
    FOREIGN KEY (manager_id) REFERENCES Employees(emp_id) ON DELETE RESTRICT
);
```

4.2 Data Manipulation Language (DML) & DQL

```
-- Inserting Records
INSERT INTO Departments (dept_id, dept_name, location) VALUES
(101, 'Engineering', 'Building A'),
(102, 'Finance', 'Building B'),
(103, 'Marketing', 'Building C');

-- Update & Delete Queries
UPDATE Employees
SET salary = salary * 1.10
WHERE dept_id = 101 AND hire_date < '2022-01-01';

DELETE FROM Employees
WHERE salary < 30000;
```

4.3 Complex Queries & Window Functions

SQL window functions compute values across sets of rows related to the current row without collapsing rows into a single summary output.

```
-- Query using Window Functions (Dense Rank & Running Total)
SELECT
    emp_id,
    first_name,
    dept_id,
    salary,
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_salary_rank,
    SUM(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS running_dept_payroll
FROM Employees;
```

4.4 SQL Joins Summary

Join Type	Result Set Description
INNER JOIN	Returns only tuples that have matching join keys in both left and right relations.
LEFT OUTER JOIN	Returns all tuples from left table; matches from right table (NULL filled if no match).
RIGHT OUTER JOIN	Returns all tuples from right table; matches from left table (NULL filled if no match).
FULL OUTER JOIN	Returns all records when there is a match in either table. Non-matching attributes are NULL.
CROSS JOIN	Produces the Cartesian product of both tables ($M \times N$ rows).

Module 5: Relational Database Design & Normalization

5.1 Redundancy & Update Anomalies

In an un-normalized schema, data duplication leads to three severe update anomalies:

- **Insertion Anomaly:** Inability to insert information about an entity without adding unrelated data for another entity (e.g., cannot create a new department unless at least one employee is assigned to it).

- **Deletion Anomaly:** Deleting a record to remove one entity inadvertently destroys data for an entirely separate entity (e.g., deleting the last employee in a department removes department records).
- **Update Anomaly:** Modifying a duplicated attribute (e.g., Department Location) requires updating thousands of identical rows. If one row is missed, data consistency is destroyed.

5.2 Functional Dependencies (FDs) & Attribute Closure

A Functional Dependency $X \rightarrow Y$ means that attribute set X uniquely determines attribute set Y across all possible valid relation instances.

Attribute Closure Algorithm (X^+):

Algorithm to Compute Attribute Closure (X^+) under FDs F

1. Initialize $X^+ = X$.
2. Repeatedly search for any FD $A \rightarrow B$ in F such that $A \subseteq X^+$.
3. Add B to X^+ (i.e., $X^+ = X^+ \cup B$).
4. Repeat until no new attributes can be added to X^+.

Candidate Key Test: If X^+ contains all attributes of relation R, then X is a **Super Key**. If no proper subset of X is a super key, X is a **Candidate Key**.

5.3 Formal Definitions of Normal Forms

Normal Form	Formal Condition for Non-Trivial FD $X \rightarrow Y$	Key Objective
1NF	All attribute domains are atomic. No repeating groups or arrays.	Eliminate non-atomic data.
2NF	Must be in 1NF, and no non-prime attribute is partially dependent on any candidate key (i.e., $Proper\ Subset\ of\ CK \rightarrow Non\text{-}Prime$ is forbidden).	Eliminate Partial Dependencies.
3NF	Must be in 2NF, and for every non-trivial FD $X \rightarrow Y$: • X is a **Super Key**, OR • Y is a **Prime Attribute** (member of a candidate key).	Eliminate Transitive Dependencies ($Non\text{-}Prime \rightarrow Non\text{-}Prime$).
BCNF	For every non-trivial FD $X \rightarrow Y$: • X MUST be a **Super Key**.	Eliminate all remaining functional dependency anomalies.
4NF	Must be in BCNF. For every non-trivial Multi-Valued Dependency (MVD) $X \twoheadrightarrow Y$, X is a super key.	Eliminate Multi-valued Dependencies.
5NF (PJNF)	Relation cannot be decomposed further without loss of data; every join dependency is implied by candidate keys.	Eliminate Join Dependencies.

5.4 Lossless-Join & Dependency Preservation

- **Lossless Join Property:** Decomposition of R into R_1 and R_2 is lossless if and only if:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{OR} \quad (R_1 \cap R_2) \rightarrow R_2$$

That is, the common attribute must be a key in at least one of the decomposed relations.

- **Dependency Preservation:** A decomposition preserves dependencies if $(F_1 \cup F_2)^+ = F^+$. It ensures FDs can be verified without performing joins.

Module 6: File Organization, Indexing & B/B+ Trees

6.1 Physical File Organizations

- **Heap (Unordered) Files:** Records are inserted at the end of the file in arrival order. Insertion is $O(1)$, but searching requires an expensive linear scan $O(N)$.
- **Sequential (Ordered) Files:** Records are sorted by a specific search key. Binary search reduces lookup cost to $O(\log_2 N)$, but insertions/deletions require expensive record shifting.
- **Hashed Files:** A hash function maps search keys directly to block addresses. Lookup is $O(1)$ for equality searches, but range queries are inefficient.

6.2 Index Taxonomy

Index Class	Ordering of Data File	Index Key Properties & Characteristics
Primary Index	Ordered by Primary Key	Defined on the ordering key field. Sparse (one index entry per block).
Clustering Index	Ordered by Non-Key Field	Defined on ordering non-key attribute. Sparse (one index entry per distinct key value).
Secondary Index	Unordered File	Defined on non-ordering attributes. Must be Dense (entry for every single record).

6.3 B-Trees vs. B+ Trees Architecture

B-Trees and B+ Trees are multi-level dynamic search trees that adapt automatically to insertions and deletions.

Architectural Structural Comparison

Feature	B-Tree	B+ Tree
Data Pointer Storage	Stored in BOTH internal nodes and leaf nodes.	Stored EXCLUSIVELY in leaf nodes. Internal nodes contain search keys & child pointers only.
Leaf Node Links	Leaf nodes are isolated / disconnected.	Leaf nodes are connected via a doubly linked list (enables fast sequential range scans).
Fan-out / Capacity	Lower fan-out because internal nodes carry data pointers.	Higher fan-out because internal nodes carry only keys; results in shorter tree height.
Search Time	Variable (can find keys at internal nodes early).	Uniform lookup time $O(\log_m N)$ as all search paths end at leaves.

Module 7: Transaction Processing & ACID Properties

7.1 Concept of a Transaction

A transaction is an execution of a program that reads or updates data items in a database. It forms a single logical unit of work (LUW).

7.2 ACID Properties Deep Dive

- **Atomicity:** "All-or-Nothing" property. Executed via log UNDO mechanisms. If a system failure happens mid-transaction, all partial updates are undone.
- **Consistency:** The database transitions from one valid state (satisfying all domain, entity, foreign key constraints) to another valid state.
- **Isolation:** Executed via Concurrency Control components. Concurrent execution of transactions produces states equivalent to executing transactions serially.
- **Durability:** Executed via log REDO mechanisms. Once committed, updates persist permanently in non-volatile storage across system crashes.

7.3 Transaction Lifecycle State Diagram

A transaction moves through five distinct states during its lifecycle:

1. **Active:** Initial state; transaction executes read and write operations.
2. **Partially Committed:** Reached after the final statement has executed, but updates are still in volatile RAM buffers.
3. **Committed:** Reached after all log records are successfully flushed to disk (WAL rule). Transaction is complete.
4. **Failed:** Entered when an error or system crash occurs during active or partially committed states.
5. **Aborted:** Entered after rollback operations complete and the database is restored to its pre-transaction state.

Module 8: Concurrency Control Mechanisms

8.1 Concurrency Anomalies

Uncontrolled concurrent execution of transactions introduces four major anomalies:

- **Dirty Read (Uncommitted Dependency):** T2 reads data written by T1 before T1 commits. T1 then aborts, leaving T2 with invalid data.
- **Lost Update:** T1 and T2 read the same data item X. Both calculate updates, but T2 overwrites T1's update without incorporating T1's change.
- **Non-Repeatable (Fuzzy) Read:** T1 reads X. T2 updates X and commits. T1 re-reads X and sees a different value within the same transaction.
- **Phantom Read:** T1 scans a set of rows matching a predicate. T2 inserts new matching rows and commits. T1 re-scans and observes newly added "phantom" rows.

8.2 Conflict Serializability & Precedence Graphs

Two operations in a schedule conflict if they belong to different transactions, operate on the exact same data item, and at least one is a Write ($W(X)$).

> **Algorithm: Serialization Graph (Precedence Graph) Test**
>
> 1. Create a node for each active transaction T_i in schedule S.
> 2. Draw a directed edge $T_i \rightarrow T_j$ if T_i performs an operation O_1 on data item X before T_j performs conflicting operation O_2 on X.
> 3. **Decision Rule:** If the Precedence Graph contains **NO CYCLES**, the schedule is Conflict Serializable. Topological sort gives equivalent serial order.

8.3 Lock-Based Protocols & 2PL

- **Basic Two-Phase Locking (2PL):** Guarantees conflict serializability.
  - *Growing Phase:* Transaction acquires locks, cannot release any locks.
  - *Shrinking Phase:* Transaction releases locks, cannot acquire new locks.
- **Strict 2PL:** Transaction holds all Exclusive (X) locks until it commits or aborts. Prevents cascading rollbacks.
- **Rigorous 2PL:** Transaction holds ALL locks (both Shared and Exclusive) until commit/abort. Ensures strict serializability.

8.4 Deadlock Prevention & Detection

Deadlock occurs when transactions form a circular wait for locks.

- **Wait-Die Scheme (Non-preemptive):** Uses timestamps $TS(T)$. If older T_i requests lock held by younger T_j, T_i waits. If younger T_j requests lock held by older T_i, T_j dies.
- **Wound-Wait Scheme (Preemptive):** If older T_i requests lock held by younger T_j, T_i preempts (wounds) T_j, forcing it to abort. If younger requests from older, younger waits.

Module 9: Database Recovery Systems

9.1 Write-Ahead Logging (WAL) Protocol

Recovery managers maintain an append-only transaction log recording state changes. WAL mandates:

1. Log records describing a data modification must be flushed to stable disk media BEFORE dirty data blocks are written to disk.
2. All log records for a transaction must be flushed to disk BEFORE the transaction is granted commit status.

9.2 Checkpointing

Checkpoints reduce recovery overhead by truncating the portion of log that needs processing after a crash. During a checkpoint:

- All modified RAM log buffers are flushed to disk.
- All modified dirty database pages in RAM buffers are written to disk.
- A `<CHECKPOINT L>` record listing active transactions `L` is logged.

9.3 ARIES Recovery Algorithm Overview

Algorithms for Recovery and Isolation Exploiting Semantics (ARIES) is the industry standard for DBMS recovery. It executes 3 phases post-crash:

1. **Analysis Phase:** Scans log forward from last checkpoint to identify dirty pages in buffer pool and active transactions at crash time.
2. **REDO Phase:** Scans forward from history point to re-apply modifications of ALL transactions (committed or uncommitted) to restore exact pre-crash state.
3. **UNDO Phase:** Scans backward, rolling back actions of transactions that were active (uncommitted) at the crash point.

Module 10: Advanced Topics & Distributed Systems

10.1 Two-Phase Commit Protocol (2PC)

Ensures atomic multi-site transaction commit in distributed databases:

- **Phase 1 (Prepare):** Coordinator asks participants `PREPARE`. Participants write WAL log records and respond `VOTE_COMMIT` or `VOTE_ABORT`.
- **Phase 2 (Commit):** If ALL vote commit, Coordinator logs and sends `GLOBAL_COMMIT`. If ANY vote abort, Coordinator sends `GLOBAL_ABORT`.

10.2 CAP Theorem & NoSQL Data Models

The CAP Theorem states that a distributed data store can simultaneously guarantee at most two of three properties: **Consistency**, **Availability**, and **Partition Tolerance**.

NoSQL Class	Architecture & Format	Best Use Cases
Document Store	Stores semi-structured documents (JSON/BSON/ XML). Flexible schemas.	Content Management, E-commerce Product Catalogs (MongoDB, CouchDB)

Key-Value Store	Simple hash table mapping unique keys to arbitrary binary payloads.	Caching, Session Storage, High-speed Queues (Redis, Amazon DynamoDB)
Column-Family	Stores data in sparse columns rather than rows. Optimized for fast analytics.	Time-series Data, Big Data Analytics (Apache Cassandra, HBase)
Graph Database	Nodes, Edges, Properties representing highly connected network structures.	Social Networks, Recommendation Engines, Fraud Detection (Neo4j)

Appendix A: DBMS Review & Exam Technical Questions

Question 1: Differentiate between 3NF and BCNF with a concrete example.

Answer: A relation is in 3NF if for every FD $X \rightarrow Y$, X is a super key OR Y is a prime attribute. BCNF removes the "prime attribute" relaxation, requiring X to ALWAYS be a super key.

Example: Consider `Advisor(student_id, advisor_name, subject)` where a student has one advisor per subject, but an advisor teaches only one subject. FDs are: $\{student_id, subject\} \rightarrow advisor_name$ and $advisor_name \rightarrow subject$. Candidate keys are $\{student_id, subject\}$ and $\{student_id, advisor_name\}$. This is in 3NF (because $subject$ is prime), but NOT in BCNF because $advisor_name$ is not a super key.

Question 2: What is the Cascading Rollback problem and how does Strict 2PL solve it?

Answer: Cascading rollback occurs when an uncommitted transaction T_1 aborts, forcing a chain reaction of rollbacks across other dependent transactions (T_2, T_3) that read dirty data written by T_1. **Strict 2PL** prevents cascading rollbacks by mandating that transactions hold all Exclusive (X) locks until final commit/abort, preventing any other transaction from reading uncommitted modifications.

Question 3: Why are B+ Trees preferred over B-Trees for database indexing?

Answer: 1. Higher Fan-out: B+ Tree internal nodes store only key search values and pointers (no data records), allowing more keys per node block and yielding a shallower tree.
2. Efficient Sequential Range Scans: B+ Tree leaf nodes form a continuous linked list. Range queries (`WHERE age BETWEEN 20 AND 50`) simply traverse leaves sequentially, whereas B-Trees require expensive in-order tree traversals across levels.

End of Comprehensive Database Management Systems (DBMS) Notes

Prepared for University Curriculum, GATE CS Preparation & Database Systems Reference.